Software Engineering Lab
UE24CS341A
MINI – PROJECT

Software Requirements Specification (SRS)

Project : College Timetable Generator
Version : 1.0
Authors : Harshitha.B.T ; PES1UG24AM118
Diya B R ; PES1UG24AM097
Gaurav S ; PES1UG24AM104
Harsha KR ; PES1UG24AM114
Date : 27-10-2026
Status : Team draft for SRS review
Repository : harshithabt/college-timetable-generator
Technology : Python, Flask, HTML, CSS, JavaScript, pytest, GitHub Actions

Revision history

Version	Date	Author	Change summary	Approval
1.0	27-10-2026	Project team	Initial project-specific SRS for the College Timetable Generator	Pending review

Approvals

Role	Name	Signature / Email	Date
Course Coordinator	Faculty and TA		Pending review
Project Team	Harshitha B.T ; Diya B.R ; Gaurav Shivakumar ; Harsha K.R.		27-10-2026

Table of Contents

1. Introduction
2. Overall description
3. External interfaces
4. System features (detailed)
5. Non-functional requirements (detailed)
6. Quality attributes & Acceptance tests
7. UML Use-Case Diagram
8. Requirements Traceability Matrix (RTM)

1. Introduction

1.1 Purpose

This SRS defines the functional and non-functional requirements for the College Timetable Generator. It describes the supported timetable input flow, input validation, scheduling logic, results display, automated testing, and repository workflow.

1.2 Scope

The College Timetable Generator is a Python/Flask web application with a browser interface built using HTML, CSS, and JavaScript. A user submits supported timetable data; the application validates it, invokes the scheduler, and presents a generated timetable or feedback. The repository includes scheduler and Flask tests, configured GitHub Actions checks, and pull-request-based collaboration. Deployment configuration exists in the repository, but a working live deployment must be confirmed separately.

1.3 Audience

Student Users, Faculty Members, Project Developers, Testers, Project Maintainers, and Assessment Evaluators.

1.4 Definitions

CTG : College Timetable Generator
FR : Functional Requirement
NFR : Non-Functional Requirement
UI : User Interface
CI : Continuous Integration
CD : Continuous Delivery/Deployment
PR : Pull Request
RTM : Requirements Traceability Matrix
SRS : Software Requirements Specification.

2. Overall description

2.1 Product perspective

The application consists of Flask request handling, scheduler/constraint logic, HTML templates, CSS/JavaScript assets, and automated tests. GitHub supports source control, pull-request review, and configured automated checks.

2.2 Major product functions (detailed)

- Collect timetable inputs through a browser form
- Validate required and supported values
- Invoke the scheduler with validated data

- Generate a timetable subject to implemented constraints
- Handle invalid or unschedulable requests with feedback
- Display generated results in a structured view
- Allow revised inputs and another generation request
- Run automated scheduler and Flask tests
- Run configured GitHub Actions checks
- Support branches, pull requests, reviews, and merges

2.3 User roles and characteristics

- **Student / faculty user** : Enters supported data, requests a timetable, reads results, and responds to validation feedback.
- **Project developer:** Maintains scheduler, Flask routes, templates, styles, JavaScript, and tests.
- **Project maintainer / reviewer :** Reviews changes, checks automated test results, and integrates approved changes.
- **Assessment evaluator :** Reviews the product, requirements, tests, repository workflow, and documentation.

2.4 Operating environment

Python and Flask, browser UI using HTML/CSS/JavaScript, dependencies declared in requirements.txt, and a modern desktop browser. Verify exact setup/run commands on the submitted revision.

2.5 Constraints

- Timetable output is limited by supported inputs and implemented scheduler constraints.
- Not every possible input combination is guaranteed to be schedulable.
- The application depends on declared Python packages.
- Tests and CI results apply to the revision tested.
- Deployment configuration does not prove a live deployment is working.
- Do not claim ERP integration, authentication, or notifications unless implemented.

3. External interface requirements

3.1 User interfaces

Browser-based input form and results view using HTML templates, CSS, and JavaScript. Verify actual fields and controls against the final application.

3.2 Hardware interfaces

No dedicated hardware interface is required; a computer running Python/Flask and a modern browser is sufficient.

3.3 Software interfaces

Flask connects browser requests to the scheduler. pytest runs automated tests; GitHub Actions runs configured checks. GitHub branches and pull requests support collaboration. No college ERP or external timetable API is specified.

3.4 Communications

Local development uses browser-to-Flask HTTP. Public deployment should use appropriate secure hosting configuration, including HTTPS. Record a deployment URL only after verifying it.

4. System features (detailed)

Each requirement below includes acceptance criteria and a reference test case. IDs follow ATM-F-###.

4.1 Timetable Input and Validation

Description: Collect and validate the timetable data supported by the application before scheduling.

Req ID	Requirement	Type	Priority	Source/Stakeholder	Acceptance criteria/Test case ref	Comments/ Dependencies
CTG-FR-001	The system shall provide a browser-based form for entering supported scheduling data.	Functional	High	Input form	Required fields are displayed and can be submitted.	Browser input form
CTG-FR-002	The system shall accept supported subject and timetable constraint inputs represented by the implemented form.	Functional	High	Form / scheduler	A representative valid input is accepted.	Fields must match implementation
CTG-FR-003	The system shall validate required fields before timetable generation is attempted.	Functional	High	Flask validation	Missing required data produces visible feedback.	Validation tests
CTG-FR-004	The system shall validate supported input values and handle malformed or inconsistent values safely.	Functional	High	Validation layer	Invalid data is rejected or clearly flagged.	Do not silently accept invalid values

4.2 Timetable Scheduling and Constraint Handling

Description: Generate a timetable using the scheduler and the constraints implemented in the project.

Req ID	Requirement	Type	Priority	Source/Stakeholder	Acceptance criteria / Test case ref	Comments / Dependencies
CTG-FR-005	The system shall pass validated scheduling data to the timetable-generation logic.	Functional	High	Scheduler integration	A valid request invokes the scheduler.	Scheduler module
CTG-FR-006	The scheduler shall generate a timetable while respecting the hard constraints implemented in the project.	Functional	High	Scheduler / constraints	Automated tests verify representative supported constraints.	Only implemented constraints
CTG-FR-007	The system shall handle a request for which a valid timetable cannot be produced without an unhandled application failure.	Functional	High	Scheduler / Flask	An unschedulable test case produces controlled behavior.	Verify against current code

4.3 Results Display and User Interaction

Description: Present the generated timetable and provide clear feedback during user interaction.

Req ID	Requirement	Type	Priority	Source/Stakeholder	Acceptance criteria / Test case ref	Comments / Dependencies
CTG-FR-008	The system shall return a successfully generated timetable to the browser for display.	Functional	High	Flask routes / templates	A valid request renders a results view.	Results template
CTG-FR-009	The system shall present timetable output in a structured, readable format.	Functional	High	Results template / CSS	Users can identify scheduled entries and displayed	UI review

CTG-FR-010	The system shall show user-facing feedback when input validation or timetable generation fails.	Functional	High	Flask / UI	Failure feedback is visible to the user.	Error feedback
CTG-FR-011	The system shall allow users to return to the input view and submit revised data for another generation request.	Functional	Medium	UI / navigation	A second request can be made after viewing a result.	Navigation flow
CTG-FR-012	The interface shall support the client-side interactions implemented in the project's JavaScript.	Functional	Medium	JavaScript	Implemented controls respond as expected in a browser check.	Check actual JS interactions
CTG-FR-013	The interface shall apply the styling and theme behavior implemented in the project's CSS and templates.	Functional	Medium	CSS / templates	The page renders with supported styles and theme behavior.	Check actual UI behavior

4.4 Application Execution and Project Workflow

Description: Support local execution, automated verification, and the team's repository workflow.

Req ID	Requirement	Type	Priority	Source/Stakeholder	Acceptance criteria / Test case ref	Comments / Dependencies
CTG-FR-014	The project shall provide a supported application entry point for running the Flask application locally.	Functional	High	Flask app	The documented command starts the application in the intended environment.	README / app entry point
CTG-FR-015	The project shall include automated tests covering scheduler behavior and Flask application behavior.	Functional	High	pytest / tests	The documented pytest command executes and reports results.	Run tests on final revision

CTG-FR-016	The repository shall run configured automated checks through GitHub Actions for supported repository events.	Functional	High	GitHub Actions	A qualifying event produces a workflow run and visible result.	Inspect final workflow run
CTG-FR-017	The repository shall support version control and pull-request-based review of team code changes.	Functional	Medium	Git / GitHub	Changes can be proposed, reviewed, and merged through pull requests.	PR workflow
CTG-FR-018	The project shall provide setup, run, and test instructions in its README.	Functional	Medium	README	README contains prerequisites and commands needed to run and test the project.	Verify README before submission

5. Non-functional requirements (detailed)

Numeric performance or availability guarantees have not been invented where no project target has been established.

Req ID	Category	Requirement	Acceptance criteria / measurement
CTG-NFR-001	Usability	The input and results pages shall present labels, output, and feedback in a way understandable to student or faculty users.	Manual review of the main input-to-results flow.
CTG-NFR-002	Reliability	Invalid input and tested unschedulable cases shall be handled without an unhandled application exception.	Run invalid-input and scheduler regression tests.
CTG-NFR-003	Maintainability	Scheduler logic, Flask request handling, templates, static assets, and tests shall remain identifiable components in the codebase.	Code review of the repository structure.
CTG-NFR-004	Testability	Automated tests shall be runnable using the documented pytest command.	Run `python -m pytest -v` on the submitted revision.
CTG-NFR-005	Portability	The application shall run in a supported Python environment using the dependencies declared by the project, without developer-specific absolute paths.	Install declared dependencies in a clean environment and run the documented command.

CTG-NFR-006	Performance	Timetable generation for representative project inputs should complete within a practical demonstration time; no numeric response-time target has been formally established.	Measure generation time on documented sample inputs before stating a numeric target.
CTG-NFR-007	Browser compatibility	The core form and results flow should work in a current desktop browser.	Manually test the core flow in a current browser.

5.1 Security

5.1.1 Security objectives

ID	Security objective
CTG-SO-001	Input integrity: prevent malformed or inconsistent user input from being silently treated as valid scheduling data.
CTG-SO-002	Change integrity: make code changes reviewable and run the repository's configured automated checks where applicable.

5.1.2 Security requirements

Req ID	Requirement	Type	Priority	Acceptance criteria / test
CTG-SR-001	The application shall validate user-supplied fields on the server before passing data to the scheduler.	Input validation	High	Invalid values are rejected or handled with controlled feedback.
CTG-SR-002	User-facing error messages shall not expose Python tracebacks, environment variables, or secret configuration values.	Error handling	High	Representative invalid requests do not reveal runtime details.
CTG-SR-003	Credentials, access tokens, and deployment secrets shall not be committed as plain values to the repository.	Repository hygiene	High	Review tracked files and workflow/configuration for exposed secrets.
CTG-SR-004	Shared-code changes should be submitted through pull requests and reviewed before merging according to the team's workflow.	Change control	Medium	Inspect pull-request history and applicable repository settings.

CTG-SR-005	Application logs, where used, shall not record secret values or unnecessary submitted data.	Logging / privacy	Medium	Review logging and representative error paths.

6. Quality attributes & acceptance tests

Acceptance exit criteria: core flow works for supported inputs, high-priority requirements are verified, no known critical failure remains in the tested flow, tests run on the final revision, and RTM status is evidence-based.

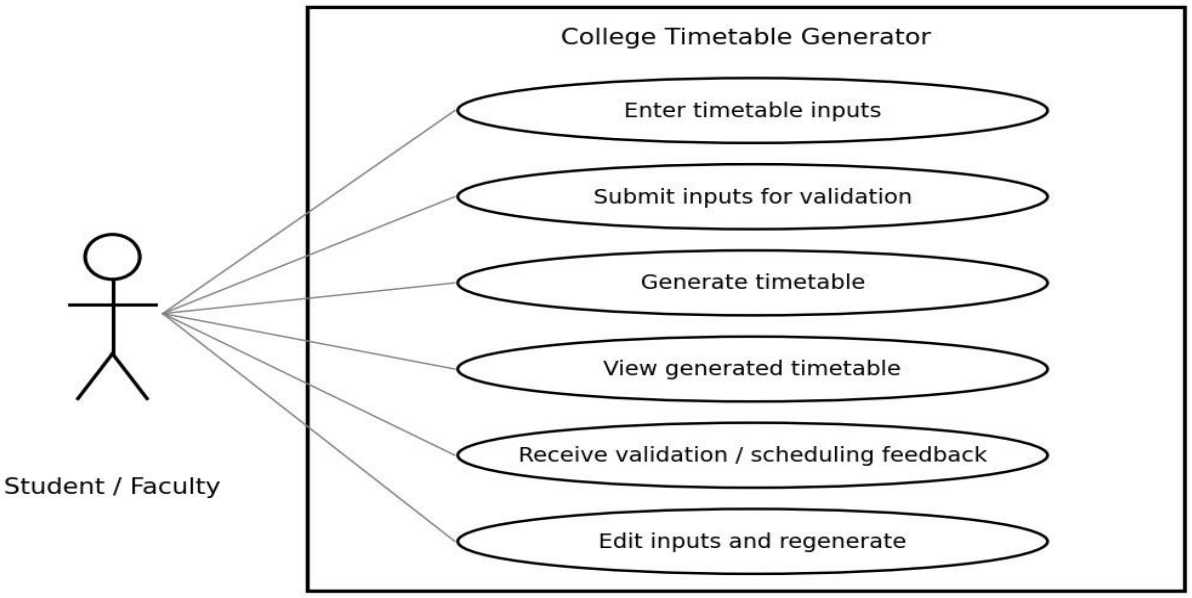
Test area	Acceptance approach	Evidence
Input validation	Test valid, missing, malformed, inconsistent inputs.	Test output/screenshots.
Scheduler constraints	Run scheduler unit/regression tests including unschedulable cases.	pytest output and test names.
Flask integration	Run Flask tests and browser input-to-results flow.	Test output/demo evidence.
Automated tests / CI	Run `python -m pytest -v` and inspect Actions.	Final test summary/workflow run.
Usability	Check labels, navigation, readable results, feedback.	Manual checklist.
Security hygiene	Review validation, errors, logs if present, repository secrets.	Review checklist; no secret values.
Deployment / E2E if required	Test core flow in target deployment.	URL and E2E evidence; only mark verified after test.

7. System models and diagrams

7.1 Timetable generation use cases (UML Use-Case diagram)

User-facing interactions with the College Timetable Generator.

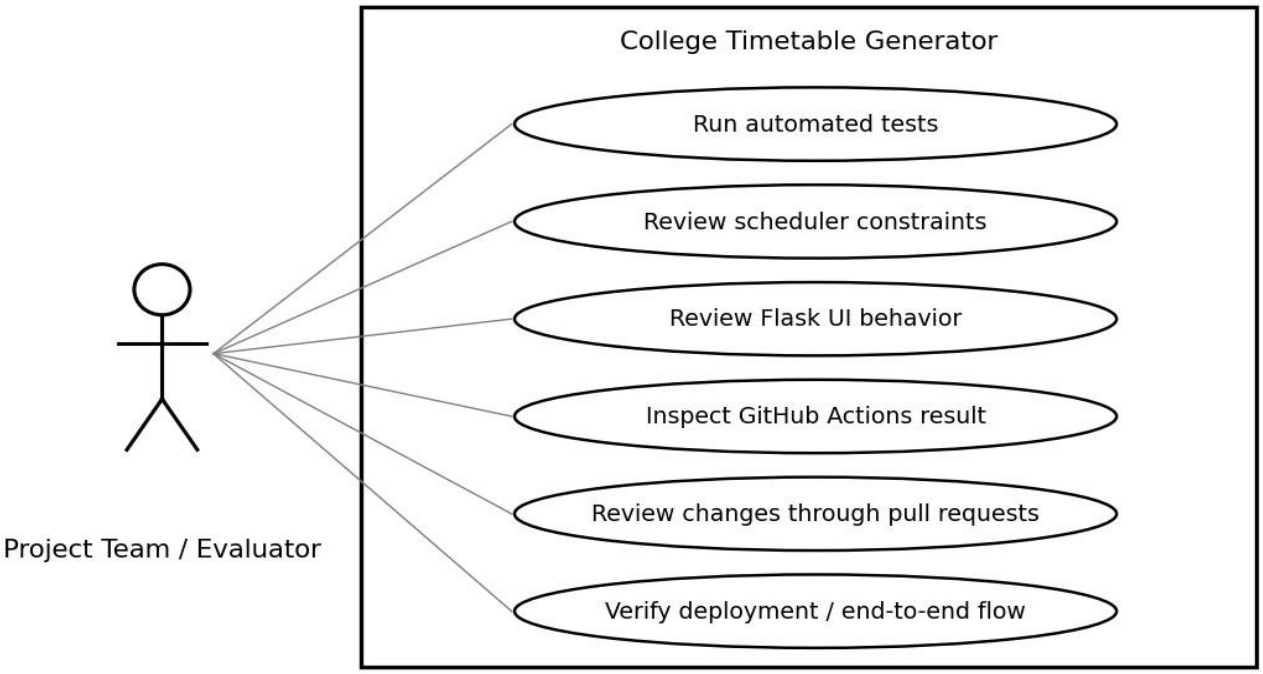
Use Case Diagram 1 — Timetable Generation



7.2 Development and verification use cases

Team verification and review activities; the deployment use case does not claim production deployment has passed.

Use Case Diagram 2 — Development and Verification



8. Requirements Traceability Matrix (RTM)

Status key: P = Passed, F = Failed, N = Not yet verified. The previous integrated test run was reported as 70 passed; rerun on the exact submission revision.

Requirement ID	Summary	Section / design	Module	Test / verification	Status / comments
CTG-FR-001–004	Input form / validation	4.1 / input handling	Form / Flask validation	Flask tests + manual checks	Passed
CTG-FR-005–007	Scheduling / constraints	4.2 / scheduler	Scheduler	Unit/regression tests	Failed CTG-NFR-007
CTG-FR-008–013	Results / UI	4.3 / UI	Routes, templates, CSS, JS	Flask tests + browser walkthrough	Passed
CTG-FR-014	Local startup	4.4 / setup	Flask entry point	Run documented command	Passed
CTG-FR-015	Automated tests	4.4 / test suite	tests/	`python -m pytest -v`	Passed: 70 tests
CTG-FR-016–017	CI / collaboration	4.4 / repo workflow	GitHub Actions / PRs	Inspect final run and PR history	Passed
CTG-FR-018	README	4.4 / documentation	README	Review setup/run/test steps	Passed
CTG-NFR-001–007	Quality attributes	5 / quality checks	UI / scheduler / repo	Manual review, tests, clean install, timing	Failed CTG-NFR-006
CTG-SO-001–002; CTG-SR-001–005	Security	5.1 / security	Validation, errors, logs, repo settings	Code/config review	Passed

~ Thank You ~